DhanDrishti

DhanDrishti is an accessibility-focused, AI-assisted banknote recognition and currency utility application. A user can scan a banknote with a phone camera or upload an image, identify its currency and denomination, view a confidence score, convert the value, save the result to account history, and receive localized voice feedback.

The project is designed to make everyday cash identification easier for people with low vision, travelers, cashiers, students, and anyone handling multiple currencies. It also provides educational security-feature checklists for manual counterfeit screening.

> **Important:** DhanDrishti is an assistive tool, not a certified banknote authentication device. A low-confidence result or counterfeit warning should be verified by a bank, authorized cash handler, or official currency verification equipment.

Project aim

The aim of DhanDrishti is to combine computer vision, accessible interaction, currency conversion, and voice assistance in one workflow:

1. Capture a clear image of a banknote.
2. Detect its currency and denomination using a trained YOLO model.
3. Show the predicted value and confidence.
4. Warn the user when the prediction is uncertain.
5. Convert the value to a selected target currency.
6. Speak the result in the user's selected language when voice is enabled.
7. Preserve a personal history of detections.
8. Guide the user through visible security features when a manual counterfeit check is required.

Main capabilities

- Camera-based scanning and image upload.
- Single-currency, multiple-currency, and automatic detection modes.
- YOLO ONNX inference on the server or directly in the browser demo.
- Detection confidence and low-confidence explanations.
- Support for 36 banknote classes across six currencies.
- Live exchange-rate conversion with indicative offline fallback rates.
- Account signup, login, profile preferences, and MongoDB history.
- Localized onboarding, result messages, and voice feedback.
- Browser speech recognition for supported navigation flows.
- Optional ElevenLabs text-to-speech and transcription through a backend proxy.
- Security-feature guides for watermark, thread, ink, hologram, microprinting, serial number, UV, and raised-print checks.
- Accessibility controls including large text, high contrast, dark mode, vibration, and automatic result announcements.

Supported currencies and denominations

The model recognizes **6 currencies** and **36 classes**:

Currency	Classes / denominations
USD - US Dollar	$1, $5, $10, $20, $50, $100
PHP - Philippine Peso	P20, P50, P100, P200, P500, P1000, P1000 New
INR - Indian Rupee	Rs10, Rs20, Rs50, Rs100, Rs200, Rs500, Rs2000
EUR - Euro	EUR5, EUR10, EUR20, EUR50, EUR100, EUR200
AUD - Australian Dollar	A$5, A$10, A$20, A$50, A$100
CAD - Canadian Dollar	C$5, C$10, C$20, C$50, C$100

The two PHP 1000 entries are separate model classes because they represent the old and new note designs. The class order in `Currency_Vision/dataset.yaml` and `Currency_Vision/src/banknotes.ts` must remain synchronized with the trained model.

Technology stack and why it is used

Technology	Use in DhanDrishti	Why it was selected
React 19	Main user interface	Component-based UI and a large ecosystem for accessible interfaces
TypeScript	Frontend, backend, and model integration	Static typing for API payloads, detection classes, and user preferences
Vite	Development server and builds	Fast local development and simple production bundling
Tailwind CSS	Frontend styling	Consistent responsive layouts and accessibility-oriented visual states
Node.js + Express	Application and inference API	Lightweight HTTP API with a familiar TypeScript runtime
YOLOv8-derived detector	Banknote object detection	Single-pass detection of note location, currency, and denomination
ONNX	Model deployment format	Portable model artifact usable in Node and browser runtimes
ONNX Runtime Node/Web	Inference execution	CPU server inference and optional browser-side WebAssembly inference
Sharp	Image preprocessing	Reliable resize, alpha removal, RGB conversion, and raw tensor preparation
MongoDB	Users and detection history	Document storage suited to account preferences and variable detection records

Technology	Use in DhanDrishti	Why it was selected
ExchangeRate API	Online currency conversion	Keyless exchange-rate lookup for supported conversions
ElevenLabs (optional)	Neural voice and transcription proxy	Natural multilingual voice output without exposing the API key to the browser
Browser MediaDevices API	Camera capture	Native camera access without a separate mobile application
Browser SpeechRecognition	Voice navigation	Hands-free navigation where the browser supports it

System workflow

```
User
  -> React/Vite interface
  -> Camera frame or uploaded image
  -> Frame quality checks
  -> POST /api/inference
  -> Sharp resize and RGB normalization to 640 x 640
  -> ONNX Runtime Node
  -> YOLO currency detector
  -> Class, denomination, and confidence filtering
  -> Result screen and optional voice announcement
  -> Currency conversion and account history
```

Detailed detection flow

1. The browser requests camera permission and captures a JPEG frame.
2. Brightness, sharpness, stability, and camera readiness are checked.
3. A browser-side ONNX validation pass confirms that a supported note is visible before the full request is sent.
4. The frontend sends the image and detection preferences to `POST /api/inference`.
5. The backend resizes the image to 640 x 640, removes alpha, converts pixels into normalized RGB planes, and creates a float32 tensor.
6. ONNX Runtime executes the model on the CPU.
7. The class index is mapped to currency and denomination.
8. Currency filters and confidence thresholds are applied.
9. The frontend displays the result, calculates conversion, optionally saves history, and speaks a localized message.

The standalone demo uses the same model through ONNX Runtime WebAssembly and does not require the Express API or MongoDB.

Model: how it works

The runtime model is `Currency_Vision/public/model.onnx`. The repository's custom configuration is `Currency_Vision/Yolov8s-SE.yaml`:

- YOLOv8s-style detector with 36 output classes.
- Squeeze-and-Excitation attention block after the SPPF stage.
- C2f convolutional blocks and a multi-scale detection head.
- 640 x 640 input images.
- Normalized RGB float32 input tensor shaped as `1 x 3 x 640 x 640`.
- Output detections containing a confidence and class index.
- Training configuration includes 50 epochs, batch size 8, image size 640, mosaic augmentation, MixUp, SGD, initial learning rate 0.01, and five frozen backbone layers in the notebook's default examples.

The included training notebook also contains an Ultralytics Hub YOLOv8m/YOLOv8s training path. The exact experiment that produced the committed ONNX artifact is not identified by a tracked run report, so the deployed artifact should be treated as the source of truth for runtime behavior.

Confidence and counterfeit threshold

There are several intentionally different thresholds:

Threshold	Value	Meaning
Model confidence floor	0.35 (35%)	Predictions below this are not accepted as normal detections
Backend default result threshold	0.65 (65%)	Used when the request does not provide a threshold
Frontend default user threshold	0.80 (80%)	User preference for accepting a clean result
Counterfeit/security review trigger	0.75 (75%)	A detected note below this confidence is routed toward a security check

The backend returns `success` when the selected result meets the request threshold, `low_confidence` when a usable result is below it, `error` when no usable detection remains, and `wrong_currency` where the UI applies a currency filter. Low confidence can be caused by poor lighting, glare, a sharp viewing angle, a folded or partially hidden note, motion blur, or an unsupported note.

The counterfeit screen is a guided manual assessment, not a trained counterfeit classifier. It combines check results such as watermark, security thread, special ink, hologram, microprinting, serial number, UV features, and intaglio/raised print into an `authentic`, `suspicious`, or `counterfeit` verdict for user guidance.

Model metrics and evaluation status

A set of tracked validation-results artifacts reports four model variants evaluated on the 36 banknote classes:

Model result	Images	Instances	Precision	Recall	F1 score*	mAP@50	mAP@50:95
YOLOv8s	742	858	0.931 (93.1%)	0.878 (87.8%)	0.904 (90.4%)	0.917 (91.7%)	0.780 (78.0%)
YOLOv8m	742	858	0.954 (95.4%)	0.900 (90.0%)	0.926 (92.6%)	0.935 (93.5%)	0.812 (81.2%)
RT-DETR	742	858	0.917 (91.7%)	0.937 (93.7%)	0.927 (92.7%)	0.937 (93.7%)	0.811 (81.1%)
YOLOv8s-SE (proposed)	759	863	0.621 (62.1%)	0.670 (67.0%)	0.645 (64.5%)	0.705 (70.5%)	0.553 (55.3%)

* F1 is calculated from each reported aggregate precision and recall using `2PR / (P + R)`. The results are object-detection metrics, not classification accuracy. The screenshots do not identify which training run produced the committed `public/model.onnx` artifact, so they should not be presented as a verified benchmark of the deployed file without matching run metadata.

Model comparison

The recorded results show that the proposed YOLOv8s-SE experiment did not outperform the baseline models in this evaluation:

- **Best precision:** YOLOv8m at **95.4%**
- **Best recall:** RT-DETR at **93.7%**
- **Best mAP@50:** RT-DETR at **93.7%**
- **Best mAP@50:95:** YOLOv8m at **81.2%**
- **YOLOv8s-SE:** 62.1% precision, 67.0% recall, 70.5% mAP@50, and 55.3% mAP@50:95

By **mAP@50**, the ranking is RT-DETR (93.7%), YOLOv8m (93.5%), YOLOv8s (91.7%), then YOLOv8s-SE (70.5%). By the stricter **mAP@50:95** metric, the ranking is YOLOv8m (81.2%), RT-DETR (81.1%), YOLOv8s (78.0%), then YOLOv8s-SE (55.3%). The proposed SE model should not be described as the best-performing model based on these recorded results.

Metric	Value in this repository	What it measures
Classification accuracy	Not reported	Correct class predictions / evaluated predictions
Precision	See comparison table above	Fraction of predicted notes that are correct
Recall	See comparison table above	Fraction of ground-truth notes detected
F1 score	See comparison table above	Harmonic mean of aggregate precision and recall
ROC curve / ROC-AUC	Not reported	Threshold trade-off between true and false positive rates
mAP@50	See comparison table above	Detection average precision at IoU 0.50
mAP@50:95	See comparison table above	Detection average precision across IoU 0.50-0.95

Metric	Value in this repository	What it measures
Validation images	742 or 759, depending on model run	Images included in each tracked validation summary
Validation instances	858 or 863, depending on model run	Annotated banknote instances in each tracked validation summary
Confusion matrix	Not committed	Per-class confusion, especially similar denominations
Inference latency	Runtime logs only	Preprocessing and model timings are logged by the API

“ROU” is not a standard metric for object detection; if it refers to recall, it is covered above. If it refers to another project-specific measure, it should be added after the exact definition and evaluation output are supplied.

The reported precision, recall, and mAP values are **object-detection** metrics, not ordinary image-classification accuracy. Classification accuracy and ROC-AUC require a defined classification evaluation protocol and saved ground-truth/prediction pairs. They are not recoverable reliably from the aggregate YOLO summary images alone. Do not describe 93.5% mAP@50 as 93.5% classification accuracy.

For an interview or project presentation, use this wording:

> “Because DhanDrishti is an object-detection system, we evaluated it using precision, recall, and mean Average Precision rather than a single classification-accuracy percentage. YOLOv8m achieved 95.4% precision, 90.0% recall, 93.5% mAP@50, and 81.2% mAP@50:95. RT-DETR achieved the highest recall and mAP@50 at 93.7%, while the proposed YOLOv8s-SE experiment achieved 62.1% precision, 67.0% recall, 70.5% mAP@50, and 55.3% mAP@50:95 in the recorded evaluation.”

Recommended evaluation procedure

To publish defensible model results, evaluate the exact `public/model.onnx` artifact on a held-out test set that has no image leakage from training:

1. Run YOLO validation on every one of the 36 classes.
2. Record per-class precision, recall, F1, AP@50, and AP@50:95.
3. Generate a normalized confusion matrix.
4. Report macro and weighted averages, not only overall averages.
5. Sweep confidence thresholds and record ROC-AUC or precision-recall curves.
6. Measure CPU and browser latency separately, including preprocessing.
7. Test difficult conditions: glare, blur, rotation, folds, occlusion, multiple notes, and different backgrounds.

Datasets used for training

The expected YOLO dataset is described by `Currency_Vision/dataset.yaml`:

```
path: ./data
train: images/train
```

```
val: images/val
nc: 36
```

The repository does not include the image and label files under `data/`. Consequently, the exact number of training images, validation images, test images, and class distribution cannot be verified from this checkout. The dataset must be supplied separately in YOLO format with matching image and label folders. Confirm image licensing and banknote-image redistribution rights before training or publishing a model.

Repository layout

```
.
├── README.md
├── QUICKRUN.md
├── PITCH.md
├── Currency_Vision/
│   ├── public/model.onnx          # Deployed ONNX detector
│   ├── public/samples/            # Optional demo images
│   ├── src/banknotes.ts           # Model class and conversion map
│   ├── src/inference.ts           # Browser inference helpers
│   ├── src/main.ts                # Standalone demo entry point
│   ├── server.ts                  # Express API and Node inference
│   ├── dataset.yaml               # 36-class YOLO dataset definition
│   ├── Yolov8s-SE.yaml            # Custom model configuration
│   ├── requirements.txt            # Python training dependencies
│   └── All_Model_Training_Code (1).ipynb
└── Frontend/
    ├── src/pages/                 # Application screens
    ├── src/components/            # Shared UI components
    ├── src/services/              # Inference, conversion, voice, and TTS
    ├── src/context/               # Application and voice state
    └── vite.config.ts             # Port and API proxy configuration
```

Prerequisites

- Windows, macOS, or Linux.
- Node.js and npm (a current LTS release is recommended).
- MongoDB locally or a reachable MongoDB deployment for accounts and history.
- A modern browser with camera permission support.
- HTTPS or a secure local context for camera access. The frontend's basic SSL Vite plugin provides HTTPS during local development.
- Optional: an ElevenLabs API key for voice synthesis/transcription.
- Optional: Python and the packages in `Currency_Vision/requirements.txt` for model training.

Installation

Install the two JavaScript projects independently:

```
cd Currency_Vision
npm install

cd ..\Frontend
npm install
```

The repository root does not own the application scripts. Run each command from the directory containing the relevant `package.json`.

How to run the backend and frontend

Full DhanDrishti application

Create `Currency_Vision/.env` when optional services are needed:

```
PORT=8787
MONGODB_URI=mongodb://127.0.0.1:27017
MONGODB_DB=currency_vision
ELEVENLABS_API_KEY=replace-with-your-server-side-key
```

Start the backend in terminal 1:

```
cd Currency_Vision
npm run api
```

The API listens on `http://localhost:8787`. Check it with:

```
Invoke-RestMethod http://localhost:8787/health
```

Start the frontend in terminal 2:

```
cd Frontend
npm run dev
```

Open the HTTPS URL printed by Vite, normally `https://localhost:3000`, allow the local certificate if prompted, and grant camera permission.

Standalone browser demo

This mode runs inference in the browser and does not require MongoDB:

```
cd Currency_Vision
npm run dev
```

Upload an image or use a sample image from `Currency_Vision/public/samples/`.

Build and validate

```
cd Frontend
npm run type-check
npm run lint
npm run build
npm run preview
```

```
cd ..\Currency_Vision
npm run build
npm run preview
```

API overview

Method	Route	Purpose
GET	`/health`	API and model readiness
POST	`/api/inference`	Raw image inference
POST	`/api/auth/signup`	Create an account
POST	`/api/auth/login`	Authenticate a user
GET	`/api/auth/profile?email=...`	Load profile preferences
GET	`/api/history?email=...`	Load detection history
POST	`/api/history`	Save or update a history record
DELETE	`/api/history/:id?email=...`	Delete one history record
DELETE	`/api/history?email=...`	Delete account history
POST	`/api/tts`	Voice listing, transcription, or synthesis

For `/api/inference`, the image is sent as `image/jpeg`, `image/png`, or `application/octet-stream`. Optional request headers include:

- `X-Detection-Currency`
- `X-Detection-Currencies`
- `X-Detection-Mode`: `single`, `multiple`, or `automatic`
- `X-Confidence-Threshold`: numeric value between 0 and 1

Applications and use cases

- **Accessibility:** spoken denomination and conversion for low-vision users.
- **Travel:** identify unfamiliar foreign notes and convert their value.
- **Retail and cash handling:** provide a fast first-pass denomination check.
- **Education:** teach users how banknote security features work.
- **Personal finance:** keep a searchable history of scanned notes.
- **Multilingual assistance:** present and speak results in supported languages.
- **Prototype research:** demonstrate an end-to-end computer-vision product using a portable ONNX model.

Limitations and responsible use

- Only the six currencies and 36 classes listed above are covered.
- Notes that are new, damaged, folded, occluded, poorly lit, or outside the training distribution may be misidentified.
- Confidence is a model score, not a probability that guarantees correctness.
- The counterfeit workflow is a guided manual checklist, not forensic authentication.
- Online conversion rates can be unavailable or change over time; offline rates are indicative.
- The current profile/history routes use a client-supplied email as the scope. Add server-side sessions or signed tokens before public production use.
- Keep `ELEVENLABS_API_KEY` server-side and never commit `.env` files.
- Obtain appropriate consent and follow applicable privacy rules when storing images, account data, or detection history.

Troubleshooting

The scanner cannot reach the model API

Start `npm run api` from `Currency_Vision`, check `http://localhost:8787/health`, and confirm the frontend proxy targets port 8787. For a separately hosted API, set `VITE_API_URL` in the frontend environment and restart Vite. When it is unset, the frontend uses its Vite `/api` proxy, which forwards requests to `http://localhost:8787`.

MongoDB errors

Start MongoDB locally or set `MONGODB_URI` to a reachable deployment. Image inference itself does not require MongoDB, but signup, login, and history do.

Camera access is blocked

Use the HTTPS Vite URL, grant permission, and ensure another application is not using the camera.

Detection is empty or low confidence

Use a supported note, good lighting, a flat orientation, a steady camera, and keep the complete note inside the guide frame. Reduce glare and avoid folds or occlusion.

Voice features fail

Set a valid server-side `ELEVENLABS_API_KEY`, restart the backend, and verify that the provider account supports the requested operation.

Conclusion

DhanDrishti demonstrates how a portable YOLO/ONNX currency detector can be combined with an accessible React application, conversion tools, history, multilingual voice, and security-feature education. Its current strength is the complete user workflow from camera capture to spoken result. Before high-stakes or commercial deployment, the missing held-out evaluation metrics, larger and documented datasets, stronger authentication, privacy controls, and certified counterfeit-detection validation should be completed.

License and data notice

This repository does not define a license or distribute the training dataset. Confirm the licenses and redistribution terms for banknote images, model weights, fonts, third-party services, and all other assets before sharing or deploying the project.